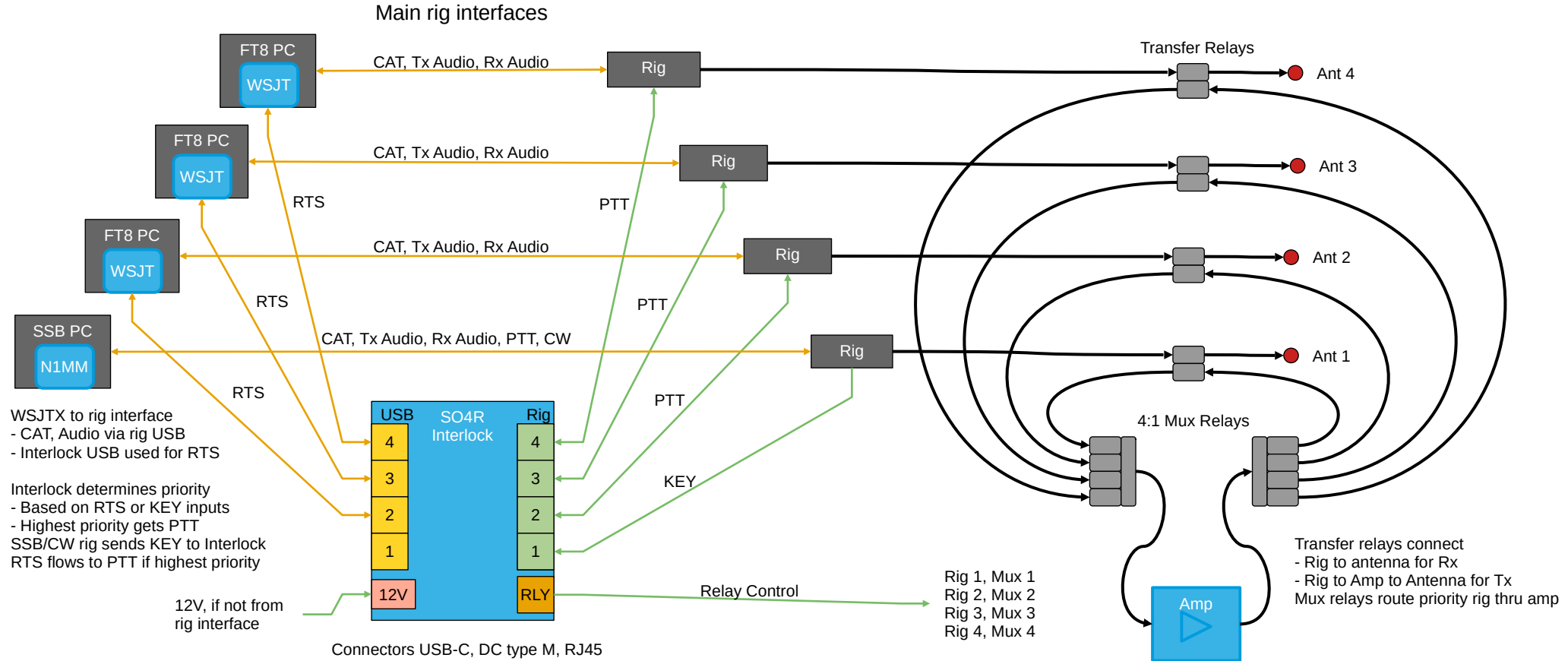


SO4R Interlock

System Engineering: Fred Lass, K2TR
Design: Jeff Millar, WA1HCO

Baseline SO4R Interlock System Diagram



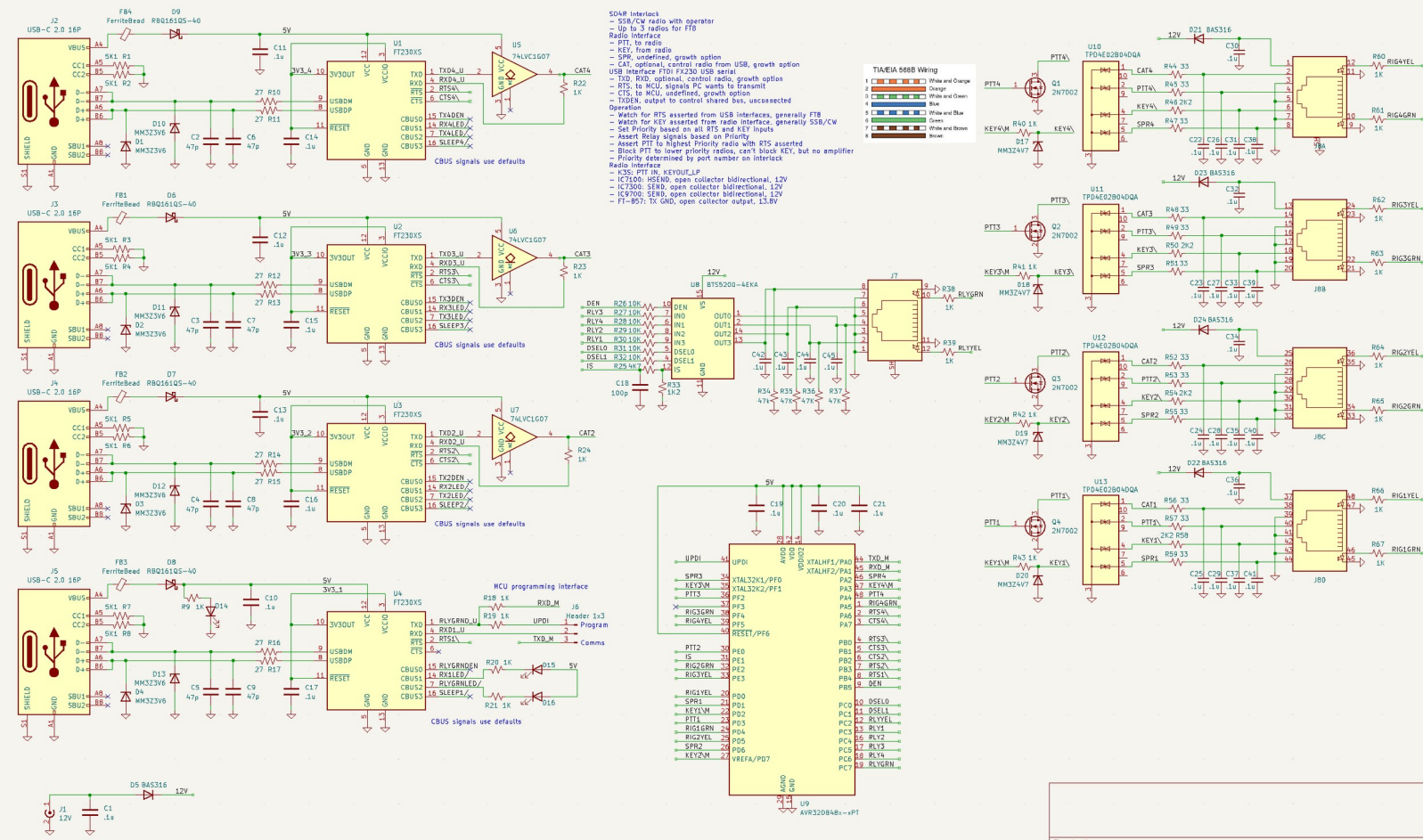
SO4R Interlock Principles of Operation

- Interlock for four radios, ensures only one Tx at a time
 - Read RTS signals from WSJTX
 - Read KEY signal from SSB/CW Rig
 - Determine Priority from RTS and KEY inputs
 - Assert PTT to the highest priority WSJT rig if no SSB/CW key
 - Repeat every 20 to 50 usec
- SSB/CW radio has highest priority
 - Tx is controlled by VOX, Footswitch, CW key, not by Interlock
 - KEY from SSB/CW rig signal tells Interlock to block PTT to other rigs
- Interlock forwards WSJTX RTS signal to rig PTT

SO4R Interlock Hardware Design

- FT230X USB serial interface
 - RTS to microcontroller
- Rig Interface, RJ45, 4 twisted pair of signal and ground
 - PTT output, open collector, 60V, 300 mA
 - KEY input, open collector, external pullup, protected to 15V
 - CAT in/out, open collector, 5V (growth option)
 - 12V, input from Rig, replaces ground on CAT pair
 - Spare (growth option)
- Relay Interface, RJ45, 4 twisted pair, relay and ground
 - Drive 12V to external relays,
 - Relay driver is BTS4200-4 with protection and diagnostics
 - Typically three Omron G2RL-1-E-12V with 80 mA draw per relay
- 12V, DC power for relays, need 240 mA
 - Used if Rig power not available
- AVR32DB48 microcontroller
 - Programmed using port 1 USB serial
 - Green LED for 5V power
 - White LEDs for serial traffic
- Electrical design
 - EMI filtering on all I/O
 - ESD protection on all inputs
 - 4 layer PCB
 - Three flooded ground planes
 - One flooded 5V plane
 - Interlock powered from any USB
 - Relays powered from any rig or 12V coaxial

Interlock Schematic Diagram



SO4R Interlock Functional Design

- MicroController
 - Reads RTS and KEY inputs
 - Writes PTT, Relay, and indicator LED outputs
- Combinatorial logic with latency below 100 usec
- Relay protection relies on Rig Tx delay
- Rig timing assumptions
 - PTT asserted to rig causes KEY output from rig immediately (usec)
 - PTT asserted to RF output delayed by 15 msec or more
 - PTT release causes RF to stop in about 300 usec (filter delays)
 - PTT release causes KEY output to be released immediately (usec)
- Conclusion, no need for sequencing within Interlock

Corner Cases

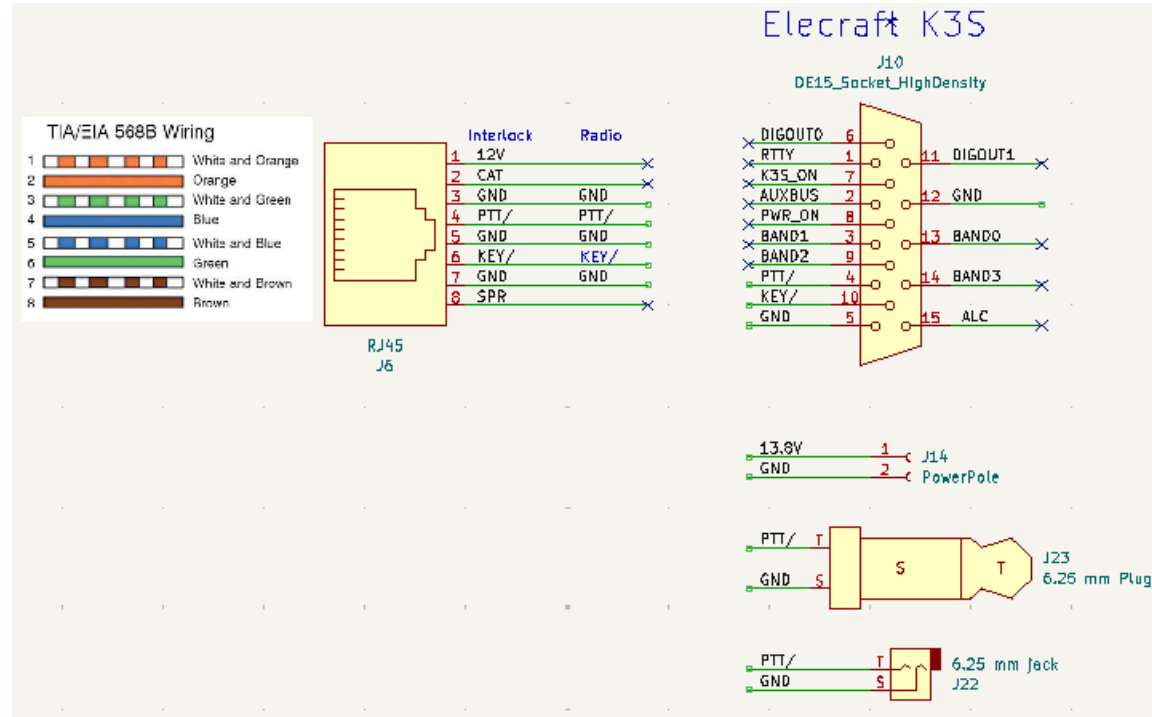
- Operator presses TX button on WSJT radio
 - LP radio _will_ go into transmit
 - Radio asserts KEY output
 - Interlock reads KEY and lights yellow LED on rig RJ45
 - Interlock does not assert relays, Tx power flows to antenna not amplifier
- If SSB/CW radio already transmitting
 - Relays set to SSB/CW radio uses amplifier
 - WSJT rig transfer relays remain in rig to antenna mode
 - LP Tx power flows to antenna
 - LP RJ45 shows Yellow LED on Interlock
- If HP radio starts Tx while LP Radio in Tx
 - Interlock sees KEY from HP radio
 - Interlock asserts relays for HP radio
 - LP radio RJ45 remains yellow
 - HP radio RJ45 shows green

SO4R Interlock LEDs

- Each RJ45 connector has a green and a yellow LED
- Rig interface
 - Green LED means radio is transmitting
 - RTS asserted, highest priority, PTT asserted, Relays asserted
 - Yellow LED means radio wants to transmit but blocked
 - RTS asserted, not highest priority, PTT blocked by Interlock
 - KEY asserted because operator pressed Transmit on radio

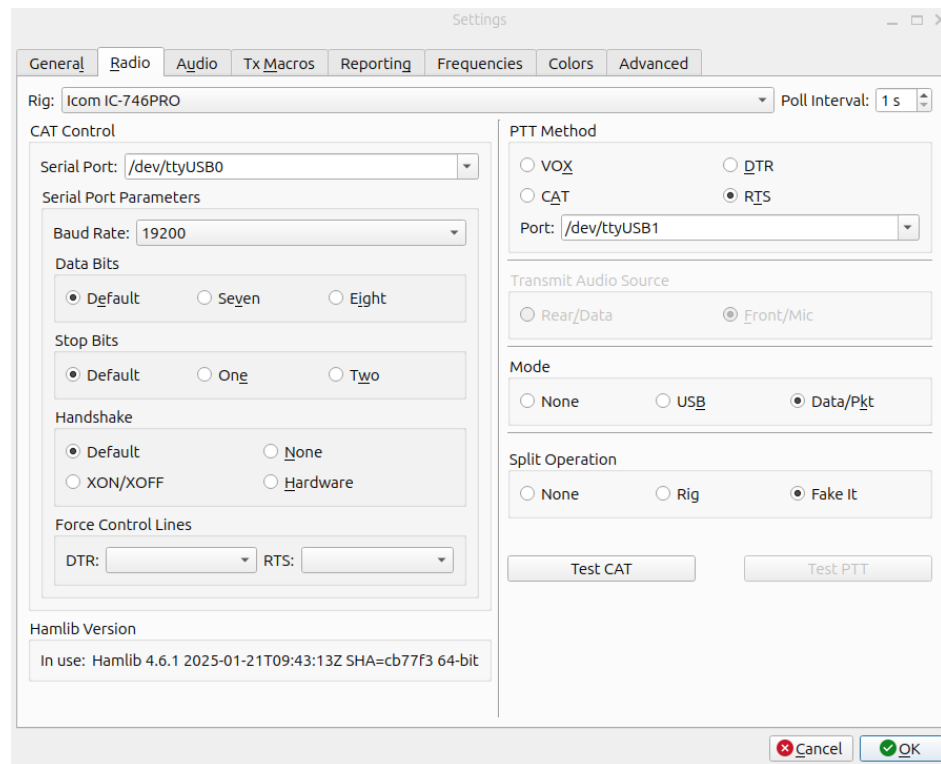
Interlock to Rig Interface Cables

- Make Cable
 - Use CAT 5/6 Ethernet cable
 - Cut in half to make two cables with RJ45
 - Expose 4 twisted pairs, strip ends
 - Solder on rig interface connectors
 - Strain Relief the junction
- Sample schematic on the right
 - Not based on experience with K3
 - Use KEY/, PTT/, and GND on DE15
 - Show ¼ inch phone jack and plug for PTT (optional)
 - Pick up 13.8V (optional)



WSJTX USB Interface from PC

- Separate USB interfaces
 - Linux WSJTX example shown
 - CAT Control USB
 - /dev/ttyUSB0
 - PTT Method USB set to RTS
 - /dev/ttyUSB1
- On Windows use separate COM ports for CAT and RTS



Identifying Interlock USB ports

- USB interfaces on Interlock programmed to make it easier to identify on a PC
 - FT230X EEPROM has reprogramed ID _strings_
 - Manufacturer: “FTDI WA1HCO”
 - Product: “SO4R Port n” where $n = \{1, 2, 3, 4\}$
- CAUTION: USB has TWO sources of Manufacturer and Product info
 - Hex codes: Vendor ID (VID) = 0x0403, Product ID (PID) = 0x6015
 - Strings: Manufacturer = FTDI WA1HCO, Product = FT230X SO4R PORT 3
 - Device Manager translates VID and PID into default string for device
 - Doesn't show the real strings within the device
 - On Windows, use USBView from Microsoft
 - Show programmed strings, used to find COM port number
 - <https://learn.microsoft.com/en-us/windows-hardware/drivers/debugger/usbview>

SO4R Interlock Options and Growth

- Rig 2, 3, 4 RJ45 connector has growth options
 - Serial Rx and Tx converted to CAT open collector
 - CAT only for Rigs 2, 3, 4. Rig 1 Serial is for programming
 - Spare input/output
- Rig 1 RJ45 used for programming, not CAT
 - Serial port used for programming MCU
 - Serial port also available for user interface
- Relay driver, BTS4200-4 is connected to MCU
 - Fault detection, open, short, etc.
 - Measure relay currents

Interlock Code (subject to change)

```
// Read all inputs, invert from low true to high true logic
bool RTS1 = !digitalRead(RTS1nPIN); // RTS from serial port asks for Tx
bool RTS2 = !digitalRead(RTS2nPIN);
bool RTS3 = !digitalRead(RTS3nPIN);
bool RTS4 = !digitalRead(RTS4nPIN);
```

```
bool KEY1 = !digitalRead(KEY1nPIN); // KEY from radio says Tx from RTS or manual
bool KEY2 = !digitalRead(KEY2nPIN);
bool KEY3 = !digitalRead(KEY3nPIN);
bool KEY4 = !digitalRead(KEY4nPIN);
```

```
// Compute Priority based on RTS and KEY inputs
// If no Tx requests, remains at 0 (Rx)
Priority = 0; // Rx mode
if (RTS4 or KEY4) Priority = 4; // WSJTX radio
if (RTS3 or KEY3) Priority = 3; // WSJTX radio
if (RTS2 or KEY2) Priority = 2; // WSJTX radio
if (RTS1 or KEY1) Priority = 1; // SSB/CW radio
```

```
// Output signals in order of timing criticality
```

```
// Assert Tx to relays
// Use priority set from RTS and KEY
// Relays asserted if either RTS or KEY input and priority
bool RLY1 = (Priority == 1);
bool RLY2 = (Priority == 2);
bool RLY3 = (Priority == 3);
bool RLY4 = (Priority == 4);
digitalWrite(RLY1PIN, RLY1);
digitalWrite(RLY2PIN, RLY2);
digitalWrite(RLY3PIN, RLY3);
digitalWrite(RLY4PIN, RLY4);
```

```
// Assert PTT to radios with RTS and corresponding to priority, all others stay Rx
// PTT asserted only on RTS and priority, do not use KEY
bool PTT1 = RTS1 and (Priority == 1);
bool PTT2 = RTS2 and (Priority == 2);
bool PTT3 = RTS3 and (Priority == 3);
bool PTT4 = RTS4 and (Priority == 4);
digitalWrite(PTT1PIN, PTT1);
digitalWrite(PTT2PIN, PTT2);
digitalWrite(PTT3PIN, PTT3);
digitalWrite(PTT4PIN, PTT4);
```

```
// Assert Rig RJ45 Green LED if rig is transmitting
digitalWrite(RIG1GRNPIN, RLY1);
digitalWrite(RIG2GRNPIN, RLY2);
digitalWrite(RIG3GRNPIN, RLY3);
digitalWrite(RIG4GRNPIN, RLY4);
```

```
// Assert Rig RJ45 Yellow LED if radio wants Tx but not highest priority
bool INH2 = (RTS2 or KEY2) and (Priority > 0) and (Priority < 2);
bool INH3 = (RTS3 or KEY3) and (Priority > 0) and (Priority < 3);
bool INH4 = (RTS4 or KEY4) and (Priority > 0) and (Priority < 4);
digitalWrite(RIG2YELPIN, INH2);
digitalWrite(RIG3YELPIN, INH3);
digitalWrite(RIG4YELPIN, INH4);
```

```
// Assert Relay RJ45 Green LED if any rig transmitting
// Assert Relay RJ45 Yellow LED if any rig inhibited
digitalWrite(RLYGRNPIN, RLY1 or RLY2 or RLY3 or RLY4);
digitalWrite(RLYYELPIN, INH2 or INH3 or INH4);
```

Github Repository

- Project has three branches, main, V1.1, and V2
 - Use V2 as of 8/25
 - https://github.com/wa1hco/SO4R_Interlock/tree/V2
- Within a branch
 - Hardware, Kicad schematics, etc
 - SO4R_Interlock
 - Radio_Interface_Cables
 - Transfer_G2RL
 - Software
 - vscode, PlatformIO, Arduino Framework
 - Readme.md